

FARO — Documentación técnica e implementación en Canvas

Libro 3 de 3 · Para quien instala, desarrolla o despliega FARO

Versión: septiembre 2026 · Reto SHIFT 2026 · Gamificación

Qué cubre este libro

Cómo está construido FARO, módulo por módulo, y cómo se conecta a un Canvas real: primero a una cuenta gratuita de Instructure (lo que se puede hacer hoy, en una tarde), después a la instancia de una institución mediante LTI 1.3 (lo que requiere que la institución participe).

Los capítulos 1 a 5 describen el código. Los capítulos 6 y 7 son procedimientos paso a paso. El capítulo 8 dice cómo comprobar que todo funciona, y el 9 qué es real y qué está simulado.

Índice

1. Arquitectura
2. Estructura del código y convenciones
3. La lógica de dominio (`src/lib/`)
4. La capa Canvas en la extensión (`src/data/canvas/`)
5. El backend (`server/`)
6. Implementación en Canvas, paso a paso: cuenta gratuita
7. Implementación en Canvas, paso a paso: institución con LTI 1.3
8. Verificación y pruebas
9. Qué es real y qué está simulado

Requisitos

Herramienta	Versión	Para qué
Node.js	22.18 o superior (o 24)	El backend corre TypeScript sin compilar (<code>node src/index.ts</code>)
npm	9+	Dependencias de la extensión
Chrome / Edge	actual	Cargar la extensión
Una cuenta de Canvas	Free-for-Teacher o institucional	Capítulos 6 y 7

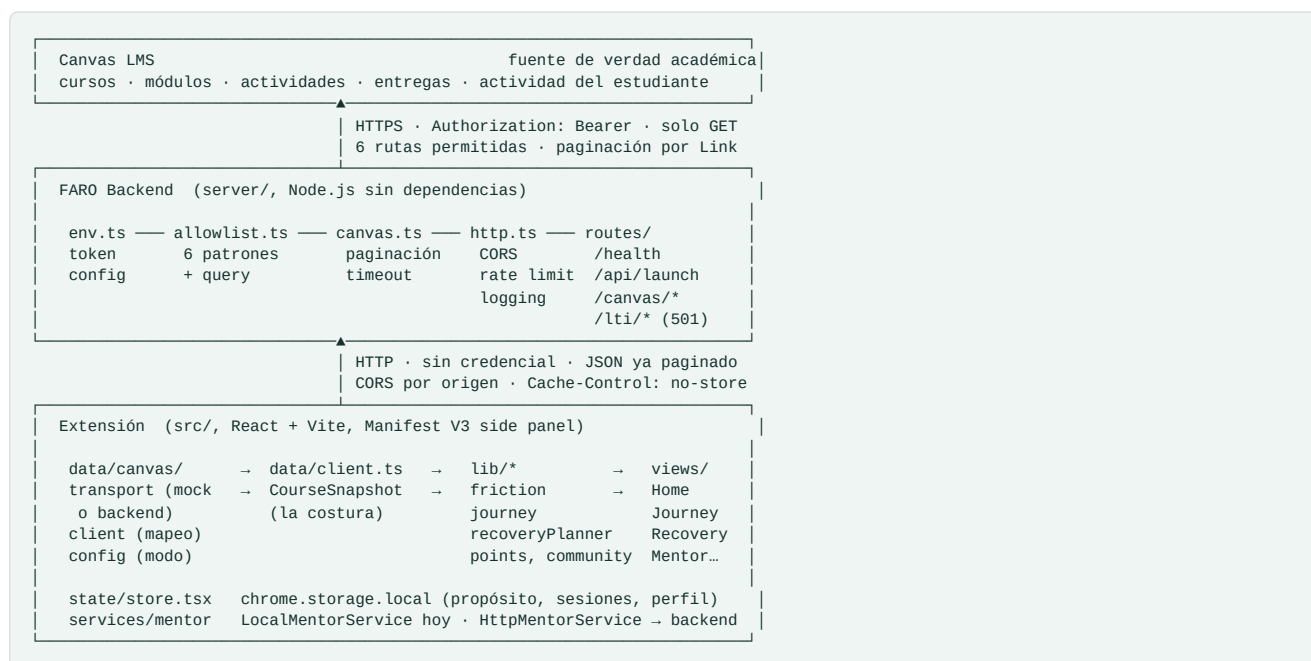
Los comandos, en una tabla

Dónde	Comando	Qué hace
raíz	<code>npm install</code>	Instala las dependencias de la extensión (68 paquetes)
raíz	<code>npm run dev</code>	Servidor de desarrollo de Vite en <code>http://localhost:5173</code>
raíz	<code>npm run typecheck</code>	Verifica tipos; si no imprime nada tras el encabezado, pasó
raíz	<code>npm run build</code>	Compila la extensión a <code>dist/</code>
<code>server/</code>	<code>cp .env.example .env</code>	Crea la configuración del backend (luego se edita)

Dónde	Comando	Qué hace
server/	<code>npm run dev</code>	Backend con recarga automática
server/	<code>npm start</code>	Backend
server/	<code>npm test</code>	20 verificaciones contra un Canvas simulado
server/	<code>npm install</code>	Solo si quieres <code>npm run typecheck</code> ahí (instala <code>typescript</code> y <code>@types/node</code> ; el backend en sí no tiene dependencias)

1. Arquitectura

1.1 Las piezas



1.2 Los cuatro principios que la forma sigue

- 1. Canvas es la fuente de verdad; FARO nunca la copia ni la altera.** Cada apertura de FARO relea el curso. No hay base de datos de calificaciones en FARO, ni la habrá: lo que FARO persiste es lo que FARO crea (propósito, sesiones, preferencias).
- 2. La credencial vive en un solo proceso.** El backend existe por esta razón. Una extensión es código público; un token dentro de ella es un token publicado. El backend guarda el token, decide qué rutas acepta y devuelve JSON. La extensión no sabe qué token hay ni cómo se llama la cabecera.
- 3. La costura es un tipo, no una clase.** `CourseSnapshot` (`src/data/client.ts`) es la única forma en que los datos del curso entran al resto de la aplicación. Todo lo que está por encima (`lib/`, `views/`) trabaja sobre ese tipo y no sabe si vino de fixtures o de una institución. Por eso cambiar de modo es una variable de entorno y no una reescritura.
- 4. La lógica no sabe que existe React.** `src/lib/` son funciones puras: entran datos, sale una decisión. Se pueden probar sin montar la interfaz, y se podrán mover al backend sin cambiarlas cuando el motor de fricción deba correr en el servidor.

1.3 Los dos modos

	mock	http
Se elige con	<code>VITE_CANVAS_MODE=mock</code> (por defecto)	<code>VITE_CANVAS_MODE=http</code>
Quién responde las rutas de Canvas	<code>MockCanvasTransport</code> con <code>fixtures.ts</code>	<code>BackendCanvasTransport</code> → backend → Canvas
Lanzamiento (curso + persona)	Fijo: curso 4021, estudiante 77120	<code>GET /api/launch</code> al backend
Perfil de estudio (ritmo medido)	Fixture: 35 min/sesión, 4 días/semana	Ausente; Recuperación lo dice
Estimación de minutos por actividad	Tabla de fixtures	20 min por defecto en todas
Necesita backend	No	Sí

Lo que **no** cambia entre modos: `client.ts` (el mapeo), `lib/` (todas las reglas), `views/` (todas las pantallas), `i18n/`. Es el mismo código contra un transporte distinto.

1.4 El flujo de una apertura

1. `main.tsx` monta `App` dentro de `StoreProvider`.
2. `store.tsx` lee `chrome.storage.local` (propósito, sesiones, preferencias, perfil).
3. `store.tsx` llama `faroClient.getCourseSnapshot()`.
4. `CanvasFaroClient` resuelve el lanzamiento (`resolveLaunch`), emite las cuatro peticiones en paralelo y mapea las respuestas a `CourseSnapshot`.
5. `store.tsx` deriva todo lo demás con `lib/`: fricción, ruta, siguiente mejor acción, plan de recuperación, puntos, logros.
6. `App.tsx` muestra Propósito si no hay destino; si no, la pestaña activa.
7. Si el paso 3 falla, `App.tsx` muestra el error con un botón *Reintentar*. Nunca se inventa un curso.

1.5 Lo que el backend hace y lo que no

Hace: guardar el token, validar rutas, hablar con Canvas, paginar, limitar tasa, restringir orígenes, registrar sin datos personales, resolver el lanzamiento en el piloto.

No hace todavía [pendiente]: persistir estado de FARO (PostgreSQL), correr el motor de fricción, llamar al modelo de IA (`POST /api/mentor` existe en la extensión como `HttpMentorService` , no en el backend), verificar lanzamientos LTI (las rutas responden 501 con la especificación en comentarios).

La regla para mover algo al backend: cuando necesite datos de más de un dispositivo, cuando necesite una credencial, o cuando deba ser auditable por la institución. Hasta entonces, se queda en la extensión.

2. Estructura del código y convenciones

2.1 El árbol

```
faro/
├── public/
│   ├── manifest.json      Manifest V3: side panel, storage; host_permissions solo localhost
│   └── background.js      service worker mínimo: abre el panel lateral
├── src/
│   ├── App.tsx            enrutado entre vistas, barra inferior, estados de carga y error
│   ├── main.tsx           punto de entrada
│   ├── types/index.ts     TODOS los tipos del dominio, en un solo archivo
│   └── data/
│       ├── client.ts      CourseSnapshot + faroClient: la costura
│       ├── canvas/        la capa Canvas (capítulo 4)
│       │   ├── endpoints.ts las 6 rutas, los scopes, el audit surface
│       │   ├── raw.ts      tipos tal como Canvas los devuelve (snake_case)
│       │   ├── transport.ts MockCanvasTransport · BackendCanvasTransport · canvasLog
│       │   ├── client.ts   CanvasFaroClient: el mapeo real
│       │   ├── config.ts   modo, URL del backend, resolución del lanzamiento
│       │   ├── status.ts   flags de comportamiento (respaldo de actividad)
│       │   └── fixtures.ts el curso de demostración, con la forma exacta de Canvas
│       └── community.mock.ts
│   ├── lib/               lógica pura, sin React ni fetch (capítulo 3)
│   ├── services/mentor.ts LocalMentorService (hoy) · HttpMentorService (backend)
│   ├── state/
│   │   ├── store.tsx      el estado de la app y sus derivaciones
│   │   ├── storage.ts     chrome.storage.local con respaldo a localStorage
│   │   ├── community.tsx  estado de Comunidad
│   │   └── theme.tsx      claro/oscuro
│   ├── views/             una vista por pantalla
│   ├── components/        piezas reutilizables (CanvasPanel, Cards, FeedPost...)
│   ├── i18n/              es.ts (fuente de verdad) · en.ts (tipado contra es) · index.ts
│   ├── styles/            tokens.css · global.css · views.css (CSS plano)
│   └── server/            el backend (capítulo 5)
│       ├── src/
│       │   ├── index.ts   arranque
│       │   ├── app.ts     ensambla rutas
│       │   ├── env.ts     configuración y .env
│       │   ├── allowlist.ts las 6 rutas permitidas
│       │   ├── canvas.ts  la única función que habla con Canvas
│       │   ├── http.ts    router, CORS, rate limit, logging
│       │   └── routes/    health · launch · canvas · lti
│       ├── test/
│       │   ├── fake-canvas.ts un Canvas de mentira con paginación real
│       │   └── e2e.ts        20 verificaciones
│       ├── .env.example
│       └── package.json    "dependencies": {}
├── docs/                 los tres libros
├── .env.example          VITE_CANVAS_MODE, VITE_FARO_API_URL
└── package.json          react, react-dom; vite, typescript
```

2.2 Las tres costuras que importan

lib/ no importa React. Si un archivo de `lib/` necesita `useState`, la lógica está en el lugar equivocado. La prueba: `src/lib/` se compila con `tsc` sin `@types/react`.

types/index.ts separa Canvas de FARO. Los tipos `Canvas*` son espejo de la API. Los demás son nuestros. Un campo que Canvas no tiene (`estimated_minutes`) lleva un comentario que lo dice.

i18n/es.ts es la fuente de verdad. `en.ts` está declarado como `const en: Dict` contra el tipo del español. Una clave añadida en un idioma y no en el otro **rompe el build**. Es intencional.

2.3 Convenciones

Convención	Regla	Por qué
Umbral	Constantes con nombre en un objeto exportado (<code>frictionRules</code> , <code>pointsConfig</code> , <code>feasibilityRules</code>)	Se ajustan en un solo lugar y se pueden mostrar en documentación
Copys	Siempre desde <code>t.*</code> (el diccionario activo), nunca literales en vistas	Dos idiomas, y el compilador vigila
Componentes	Pequeños; un <code>.tsx</code> de más de ~300 líneas probablemente contiene lógica de <code>lib/</code>	Legibilidad

Convención	Regla	Por qué
URLs	Ninguna URL de API escrita a mano fuera de <code>data/</code> y <code>server/</code>	Un solo lugar que cambiar
Comentarios	Explican <i>por qué</i> , no <i>qué</i> . Cada decisión de diseño no obvia lleva su razón junto al código	Quien llega después entiende sin preguntar
Estilos	CSS plano con tokens en <code>tokens.css</code> ; sin frameworks	Bundle pequeño; la extensión debe sentirse inmediata
Sintaxis TS en <code>server/</code>	Solo sintaxis borrrable (<code>erasableSyntaxOnly</code>): sin <code>enum</code> , sin parámetros de constructor con modificadores	Node ejecuta el TypeScript directamente sin compilar
Imports en <code>server/</code>	Con extensión <code>.ts</code> explícita	Requisito de Node para ejecutar TS

2.4 Alias

`@/` apunta a `src/` (configurado en `vite.config.ts` y `tsconfig.json`). Es un alias relativo a la raíz de Vite, por lo que funciona igual en Windows y macOS sin helpers de rutas.

2.5 Qué NO hay, a propósito

- No hay Next.js, Electron, Redux ni Tailwind. Instrucciones del proyecto: la extensión debe ser ligera.
- No hay ORM ni base de datos todavía.
- No hay framework HTTP en el backend (Fastify estaba previsto; se substituyó por `node:http` para que el proceso que sostiene el token no ejecute código de terceros; las rutas tienen la forma `(req) => reply` , que se porta a Fastify sin cambios si el proyecto lo decide).
- No hay dependencia de pruebas. Las pruebas del backend usan `node:assert` y se ejecutan con `node` .

3. La lógica de dominio (src/lib/)

Todo lo que FARO *decide* vive aquí, en funciones puras. Este capítulo documenta cada módulo con sus entradas, sus reglas exactas y sus salidas. Los números son los del código; si el código cambia, este capítulo debe cambiar con él.

3.1 friction.ts — el motor de fricción

Entrada: un `CourseSnapshot` y si el estudiante está regresando tras una ausencia. **Salida:** un `FrictionSignal` con estado, días sin actividad, pendientes, impulso y un titular de apoyo.

Umbrales (`frictionRules`, configurables):

Constante	Valor	Uso
<code>frictionDays</code>	3	A partir de aquí, FRICTION
<code>disconnectionDays</code>	7	A partir de aquí, DISCONNECTION
<code>overwhelmPendingCount</code>	4	Pendientes vencidos para POSSIBLE_OVERWHELM
<code>momentumDecayWindow</code>	14	Días de inactividad que llevan el impulso de 100 a su piso
<code>momentumFloor</code>	20	El impulso nunca baja de aquí

Estados, en orden de evaluación:

returning	→ RECOVERY
days ≥ 7	→ DISCONNECTION
pending ≥ 4 y days ≥ 3	→ POSSIBLE_OVERWHELM
days ≥ 3	→ FRICTION
en otro caso	→ FLOWING

Días sin actividad: la marca de tiempo más reciente en `snapshot.activity`. Si no hay ninguna, se asume `disconnectionDays` (7): sin evidencia de actividad, FARO trata al estudiante como desconectado, no como al corriente.

Pendientes: actividades sin entrega cuya fecha límite ya pasó.

Impulso:

```
decay = min(1, days / 14)
load  = min(1, pending / 8) × 0.2
raw   = (1 - decay) × 100 - load × 100
momentum = max(20, round(raw))
```

Un día sin actividad cuesta ~7 puntos. Ocho pendientes vencidos restan 20. Nada lo lleva a cero.

Regla de producto: el titular (`headline`) sale del diccionario por estado y es siempre lenguaje de apoyo. No existe un campo de "riesgo".

3.2 journey.ts — la ruta y la siguiente mejor acción

`buildJourney(snapshot, friction)` ordena las actividades por posición del módulo y luego por id, y asigna a cada una un estado:

tiene entrega	→ completed
sin entrega y fecha vencida	→ missed
sin entrega y módulo bloqueado	→ locked
la primera que no cae en lo anterior	→ current
las demás	→ upcoming

La última actividad de cada módulo es un **punto de control** (`checkpoint: true`). `progressPercent` = completadas ÷ total. `currentIndex` es la primera `missed` o `current`. `recalculated` es verdadero cuando el estado de fricción no es FLOWING.

`nextBestAction(journey, availableMinutes)` — la regla de cuatro pasos:

1. Abiertas = `missed` + `current` + `upcoming`.
2. De esas, las que caben en `availableMinutes`.
3. De esas, la más corta.
4. Si ninguna cabe, la más corta de todas las abiertas.

La razón mostrada depende del caso: *reabre la ruta* si es `missed`, *cabe en tu tiempo* si cabe, *la más corta* si no.

`comebackMilestone(journey)` — la abierta más corta, sin condición de tiempo. Es la misión de regreso: no la más importante, sino la que menos probabilidad tiene de posponerse otra vez.

`recoveryRoute(journey)` — tres pasos (hoy / mañana / próxima sesión) tomados de la ruta.

3.3 `recoveryPlanner.ts` — factibilidad y plan

El módulo responde con números a "¿todavía puedo terminar?".

`remainingWork(snapshot, journey)` — todo lo no completado, **incluidos los módulos bloqueados**. Ignorarlos sería la mentira tranquilizadora que este archivo existe para evitar. Devuelve actividades, minutos, módulos, días hasta el cierre y la fecha de cierre.

`capacityFor(stated, profile)` — lo que el estudiante sostiene por día. Lo que declaró gana; si no declaró nada, el ritmo medido repartido en la semana: $(\text{minutosPorSesión} \times \text{díasPorSemana}) \div 7$, redondeado a 5. Sin ninguno de los dos, 0.

`feasibility(work, capacity)` — el veredicto:

```
activities = 0                                → comfortable (terminado)
sin deadline, o daysLeft ≤ 0, o capacity ≤ 0 → unknown (no se adivina)
required = minutes / daysLeft
load      = required / capacity
load ≤ 0.85                                → comfortable
load ≤ 1.40                                → tight
en otro caso                               → not_realistic
```

`feasibilityRules = { comfortableLoad: 0.85, stretchLoad: 1.4 } . not_realistic` existe a propósito.

`strategies(work, capacity)` — tres ritmos sobre el mismo trabajo, con `PACE = { comfortable: 1, balanced: 1.35, intensive: 2 }` multiplicando los minutos diarios requeridos. Cada estrategia calcula sus días necesarios y su margen (`buffer = daysLeft - days`). **A lo sumo una** lleva `suggested: true`: la factible más cercana a la capacidad del estudiante. Sin capacidad conocida, ninguna se sugiere.

`buildPlan(journey, work, dailyMinutes, strategy)` — empaqueta el trabajo en días:

- Prioridad: primero lo `missed`; luego fecha límite más próxima; luego el orden del curso.
- Presupuesto mínimo por día: 10 minutos.
- Cargado al frente: terminar antes deja el margen *antes* del cierre.
- Una actividad más larga que un día entero se programa igual, ocupando su día.
- Lo que no cabe antes del cierre va a `overflow`. Se muestra, nunca se descarta en silencio.

`todayOneThing(plan)` — el primer ítem del primer día.

3.4 `lifeHappened.ts` — el estado de vida y la intervención

Seis estados, cada uno con un tipo de intervención y un presupuesto de minutos:

Estado	Intervención	Minutos
<code>less_time</code>	<code>shrink_session</code>	10
<code>overwhelmed</code>	<code>single_action</code>	15
<code>dont_understand</code>	<code>mentor</code>	15
<code>lost_routine</code>	<code>comeback_mission</code>	10
<code>need_break</code>	<code>pause</code>	conserva el actual
<code>ready</code>	<code>continue</code>	20

Nada aquí toca fechas ni reglas académicas. Solo cambia el tamaño y la forma del siguiente paso.

3.5 `points.ts` y `rhythm.ts` — puntos

`pointsConfig` (escala semestral):

Constante	Valor
POINTS_PER_ACTIVITY	50 por actividad completada de la ruta (entregada en Canvas o terminada en FARO)
POINTS_PER_MODULE	250 por módulo con todas sus actividades completadas
STREAK_BONUS	25 (cada 3 días consecutivos, <code>rhythmMilestoneEvery = 3</code>)
COMEBACK_BONUS	125 (una vez, si hay al menos una sesión en FARO y <code>awayGap ≥ frictionDays</code>)
REVIEW_SET_BONUS	75 por juego de tarjetas de reconexión terminado

`computePoints` también devuelve `thisWeek`: lo ganado en los últimos 7 días por acciones hechas en FARO (sesiones, tarjetas y el bono de regreso si lo ganó una de ellas).

`learningRhythm(sessions)` cuenta días consecutivos con sesión, terminando hoy. Romper la racha deja de sumar; nunca resta.

3.5b `rewards.ts` — la recompensa del semestre

`rewardsConfig` = { `SEMESTER_GOAL`: 10_000, `SEMESTER_CAP_MXN`: 200 } y tres niveles sobre una sola pista:

Nivel	Puntos
\$50 MXN	2,500
\$100 MXN	5,000
\$200 MXN	10,000

- `semesterStatus({ currentCourse, previousCourses, redemption })` → total, porcentaje (tope 100), nivel alcanzado, siguiente nivel y puntos que faltan.
- Los niveles **se desbloquean, no se compran**: alcanzar uno no descuenta nada.
- `canRedeem(tier, status)` — el nivel está alcanzado y no se ha canjeado nada este semestre. **Un canje por semestre**: canjear cierra la recompensa en ese nivel.
- `remainingCoursePoints(journey)` — lo mínimo que vale terminar el curso actual (actividades + módulos restantes; los bonos van aparte).

Los cursos anteriores del semestre viven en `data/rewards.mock.ts` (simulados, etiquetados en pantalla). Calibración del demo: 7,700 de cursos anteriores + 850 del curso actual = 8,550 (85 %); terminar las actividades y módulos restantes (1,450) lleva exactamente a 10,000.

3.5c `reviewCards.ts` — tarjetas de reconexión

- `touchedModules(journey)` — módulos con al menos una actividad completada, el más reciente primero.
- `pickReviewSet(journey, bank)` — 3 preguntas: hasta 2 del módulo más reciente, luego del anterior. Determinista. Devuelve [] si no hay suficientes (la oferta se oculta; nunca se inventan preguntas).
- `shouldOfferReview({ awayGap, frictionDays, questions, records })` — se ofrece si `awayGap ≥ 3`, hay 3 preguntas y no se completó un juego hoy.
- `reviewedToday(records)` — un juego premiado por día.

El banco (`data/reviewBank.ts`) es bilingüe y está ligado al curso del demo (`REVIEW_BANK_COURSE_ID`): un curso real sin banco no recibe tarjetas. En producción, el banco lo escribe el docente o lo genera el backend a partir de las páginas del módulo; **un token de estudiante no puede leer las preguntas de los quizzes de Canvas**, y una tarjeta de reconexión nunca debe confundirse con un quiz calificado.

3.6 `achievements.ts` — los cinco logros

Id	Condición exacta
<code>the_comeback</code>	<code>awayGap ≥ 3</code> y existe al menos una sesión
<code>back_on_track</code>	al menos 2 sesiones y <code>momentum ≥ 65</code>
<code>weathered_the_storm</code>	alguna sesión con <code>pendingAtStart ≥ 2</code>

Id	Condición exacta
smart_session	alguna sesión con fitAvailableTime
finisher	progressPercent ≥ 100

Mientras no se gana, se muestra `hint`, nunca un reproche.

3.7 sessions.ts — el registro propio de FARO

Una `StudySession` es lo que FARO anota cuando el estudiante marca un paso como hecho. Guarda minutos, cuántos pendientes había al empezar y si cupo en el tiempo declarado. `applySessions` superpone las sesiones sobre la ruta para que impulso, puntos y logros respondan al instante, sin esperar a que Canvas registre la entrega.

3.8 community.ts — orden del feed y ruteo del SOS

`rankFeed(posts, ctx)` ordena por: relevancia para el curso y módulo del estudiante, si alguien pide ayuda y no tiene respuesta, y recencia. **Ignora las reacciones a propósito.** `KIND_WEIGHT` pone `help: 5` por encima de `achievement: 2`.

`sosRouting`: `topic → mentor`, `time → recovery`, `people → community`. Tres tipos de atorado, tres sistemas.

`communityPointsConfig`: `POINTS_PER_HELPFUL_MARK: 25`, `POINTS_PER_COMPLETED_ROOM: 100`. Entrar a una sala no da puntos; terminarla sí.

3.9 mentorContext.ts — la frontera de privacidad

Una función, `buildMentorContext`, que produce los once campos del `MentorContext`. Si un dato no se construye aquí, el mentor no lo recibe. No lee `Profile`. Es el único archivo que el Libro 1 cita como contrato de datos del mentor.

4. La capa Canvas en la extensión (`src/data/canvas/`)

Siete archivos. Es la única parte de la extensión que sabe que Canvas existe.

4.1 `endpoints.ts` — el audit surface

Las rutas que FARO usa, construidas desde una función cada una:

```
course(id)           GET /api/v1/courses/:id
modules(id)          GET /api/v1/courses/:id/modules?include[]=items&per_page=100
assignments(id)       GET /api/v1/courses/:id/assignments?include[]=submission&per_page=100
userActivity(id, uid) GET /api/v1/courses/:id/analytics/users/:uid/activity
activeCourses()       GET /api/v1/courses?enrollment_state=active&per_page=100
self()               GET /api/v1/users/self (solo el backend, al resolver el lanzamiento)
```

Y dos listas para el administrador de Canvas: `requiredScopes` en el formato `url:GET|/api/v1/...` de la clave de desarrollador, y `ltiScopes` con lo que FARO pide (`contextmembership.readonly`, `result.readonly`) y lo que rechaza (`score`, `lineitem`).

`PER_PAGE = 100` es el máximo de Canvas: menos viajes, mismos datos.

4.2 `raw.ts` — la forma de Canvas

Tipos en `snake_case` que declaran **solo los campos que FARO lee**. Canvas envía muchos más; se ignoran en vez de modelarse, así una versión nueva de Canvas que añada campos no rompe esta capa. Cuando este archivo y la documentación de Canvas discrepen, la diferencia debe ser visible: por eso no se traduce al estilo del código.

Campos importantes:

- `RawModule.items` — presente cuando se pidió `include[]=items`. Cada ítem tiene `content_id` (el id de la actividad a la que apunta) y `completion_requirement.completed` (si este estudiante la cumplió).
- `RawModule.state` — `locked` | `unlocked` | `started` | `completed`. **Solo aparece si quien llama es estudiante del curso.** Un docente no lo ve.
- `RawAssignment.submission` — presente con `include[]=submission`. Un `workflow_state: 'unsubmitted'` es un stub que significa *no hecho*.
- `RawUserActivity.page_views` — un objeto cuyas claves son fechas ISO redondeadas a la hora y cuyos valores son conteos. FARO conserva las claves y descarta los valores.

4.3 `transport.ts` — cómo se hace la petición

Una interfaz, dos implementaciones:

```
interface CanvasTransport {
  readonly mode: 'mock' | 'http'
  get<T>(path: string): Promise<T>
}
```

MockCanvasTransport — busca la ruta en una lista de patrones (`fixtureRoutes` en `client.ts`), espera 140 ms simulados y devuelve el fixture. Una ruta sin patrón produce **404 en el registro**, igual que una ruta equivocada contra un Canvas real. Un error de tipo en `endpoints.ts` se ve, no se esconde.

BackendCanvasTransport — hace `fetch( ${apiUrl}/canvas${path} )` sin ninguna cabecera de autorización. El backend pagina; aquí un solo viaje es la respuesta completa. Un estado no-2xx lanza `CanvasHttpError` con el código y el `error` del cuerpo JSON del backend (`path_not_allowed`, `canvas_error`, `canvas_unreachable` ...).

canvasLog — un buffer circular de 30 registros en memoria (`at`, `method`, `path`, `mode`, `status`, `ms`, `items`, `error`). El panel de Perfil se suscribe y lo dibuja. Nunca se persiste ni se envía. Es la diferencia entre afirmar una integración y mostrarla.

4.4 `client.ts` — el mapeo

`CanvasFaroClient.getCourseSnapshot()`:

- Resuelve el lanzamiento (`courseId`, `userId`).

2. Emite las cuatro peticiones en paralelo con `Promise.all`.
3. Si la de actividad falla con 401/403, no falla todo: marca `integrationStatus.activityFallback = true` y devuelve `null` para esa parte. Cualquier otro error sí se propaga.
4. Recorre los módulos y sus ítems para construir dos mapas: `moduleOfAssignment` (de `content_id` al id de módulo, solo para ítems de tipo `Assignment` o `Quiz`) y `completedByModule`.
5. Filtra las actividades a las que algún ítem de módulo apunta. **Una actividad sin ítem de módulo no está en la ruta**: Canvas permite actividades sueltas, y ponerlas en el recorrido inventaría estructura que el docente no creó.
6. Mapea cada actividad: `submission` se conserva solo si `workflow_state !== 'unsubmitted'` y hay `submitted_at`. Es decir: **hecho es lo que la entrega dice, no la fecha límite**.
7. Mapea la actividad a eventos con marca de tiempo; o, en el respaldo, deriva eventos de las fechas de entrega del propio estudiante.
8. `estimated_minutes` sale de la función de estimación inyectada: la tabla de fixtures en modo mock, `undefined` (→ 20 min por defecto) en modo http, porque los ids de fixtures coincidirían con ids reales por casualidad.
9. `profile` (ritmo medido) solo existe en modo mock. En vivo, FARO es honesto sobre no conocer el ritmo del estudiante todavía.

Tres decisiones que un integrador descuidado haría al revés, nombradas en el encabezado del archivo: la pertenencia a módulo viene de los ítems, no de la actividad; "hecho" viene de la entrega, no de la fecha; los minutos estimados son de FARO y se dicen como tales.

4.5 `config.ts` — modo y lanzamiento

```
canvasConfig = {
  mode: VITE_CANVAS_MODE === 'http' ? 'http' : 'mock',
  apiUrl: VITE_FARO_API_URL ?? 'http://127.0.0.1:3000',
}
```

No hay campo para un token y no puede haberlo.

`resolveLaunch()` devuelve el `LaunchContext` (curso, nombre, persona, `verified`, `source`). En mock es inmediato (`source: 'fixture'`). En http llama una vez a `GET /api/launch` y lo cachea; `currentLaunch()` y `onLaunch()` permiten que el panel de Perfil se actualice cuando llega.

`verified` es `false` en ambos modos hoy. Solo un lanzamiento LTI firmado lo pondrá en `true`, y el panel lo dice.

4.6 `status.ts` — flags de comportamiento

Un objeto observable con `activityFallback`. Existe para que un respaldo no pase en silencio como si fuera el dato real: el panel de Perfil muestra la nota *"tu cuenta no puede leer las analíticas de Canvas, así que la actividad se calcula con las fechas de tus propias entregas"*.

4.7 `fixtures.ts` — el curso de demostración

Un curso en español, *Gestión de Proyectos* (GP-101), 6 módulos, 16 actividades, con la forma exacta que Canvas envía. El escenario: un adulto cinco semanas dentro de un curso, que dejó de entrar hace `DAYS_INACTIVE = 5` días, con `DAYS_UNTIL_COURSE_ENDS = 12` días para el cierre. Es el momento exacto para el que FARO existe.

Con estos fixtures los números esperados son: 16 actividades, 7 completadas (44 %), 9 restantes (4 h 35 min), 12 días, 5 sin actividad, módulo 3 iniciado, módulo 5 bloqueado, estado `FRICTION`.

4.8 Cambiar de modo

```
# .env.local en la raíz (git lo ignora)
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

`npm run dev` o `npm run build`. Ningún archivo de código cambia.

5. El backend (server/)

Un proceso Node.js, sin dependencias, que guarda el token de Canvas y sirve la API de FARO. Corre TypeScript directamente (`node src/index.ts` , Node 22.18+).

5.1 Configuración (env.ts)

Se lee una vez al arrancar, de `process.env` con respaldo en `server/.env` . El proceso **se niega a arrancar** sin `CANVAS_API_URL` ni `CANVAS_ACCESS_TOKEN` , y sin `https://` (salvo localhost).

Variable	Por defecto	Qué es
<code>CANVAS_API_URL</code>	— (obligatoria)	<code>https://canvas.instructure.com</code> o <code>https://<institución>.instructure.com</code>
<code>CANVAS_ACCESS_TOKEN</code>	— (obligatoria)	Token de la cuenta de estudiante
<code>FARO_COURSE_ID</code>	vacío	Curso a abrir; si falta, la primera inscripción activa
<code>FARO_HOST</code>	<code>127.0.0.1</code>	Interfaz de escucha
<code>FARO_PORT</code>	<code>3000</code>	Puerto
<code>FARO_ALLOWED_ORIGINS</code>	<code>http://localhost:5173,http://127.0.0.1:5173</code>	Orígenes de navegador permitidos, separados por coma
<code>FARO_RATE_LIMIT_PER_MINUTE</code>	<code>120</code>	Peticiones por minuto por cliente
<code>FARO_CANVAS_TIMEOUT_MS</code>	<code>10000</code>	Tiempo máximo de una llamada a Canvas
<code>FARO_LOG_LEVEL</code>	<code>info</code>	<code>info</code> o <code>debug</code>

`describe(env)` es la única representación de la configuración que puede imprimirse o devolverse: host de Canvas, puerto, orígenes y `tokenPresent: true` . Nunca el token.

5.2 Lista blanca (allowlist.ts)

Seis patrones, cada uno con su `purpose` escrito al lado. `allow(pathname, search)` :

1. Decodifica la ruta (`%2e%2e` no escapa).
2. Rechaza `..` y `//` .
3. Busca un patrón que coincida; si no, `path_not_allowed` .
4. Reconstruye la query solo con claves permitidas (`include[]` , `per_page` , `enrollment_state` , `page`) y valores validados (`include[]` solo `items` o `submission` ; `enrollment_state` solo `active`).
5. Devuelve la ruta exacta que se enviará a Canvas.

Este archivo **debe coincidir** con `src/data/canvas/endpoints.ts` de la extensión. Si se añade una ruta en uno, se añade en el otro, y se documenta su propósito.

5.3 La llamada a Canvas (canvas.ts)

`canvasGet(env, path)` es la única función que construye una petición hacia Canvas:

- `GET` , `Authorization: Bearer` , `Accept: application/json` , `User-Agent: FARO/0.1 (read-only)` .
- `AbortController` con el timeout configurado.
- `redirect: 'manual'` : una redirección no se sigue (podría llevar el token a otro host).
- Sigue `Link: rel="next"` hasta el final y concatena los arreglos; máximo 50 páginas.
- Antes de cada página, verifica que la URL siga en `env.canvasApiUrl` .
- Errores como `CanvasError { status, path, kind: 'http' | 'timeout' | 'network' }` . Sin cuerpo de respuesta.

5.4 La capa HTTP (`http.ts`)

`HttpApp` sobre `node:http`:

- **Router** por método y expresión regular. Ruta desconocida → `404`; ruta conocida con método incorrecto → `405`.
- **CORS**: si hay `Origin` y no está en la lista → `403 origin_not_allowed`, sin cabeceras CORS. Preflight `OPTIONS` → `204` con `GET, OPTIONS`.
- **Rate limit**: ventana fija por dirección remota; al exceder → `429` con `Retry-After: 60`. Un barrido cada minuto descarta ventanas viejas.
- **Cabeceras** en toda respuesta: `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `X-FARO-Request: <id>`.
- **Log**: una línea JSON por petición con `id`, `method`, `path` (sin query), `status`, `ms`. Los errores internos se registran con su mensaje y el cliente recibe `500 internal`, nunca el detalle.

Los handlers tienen la forma `(req: Req) => Reply | Promise<Reply>`, con `json(status, body, headers?)` como ayudante. Esa forma se porta a Fastify sin cambios.

5.5 Rutas (`routes/`)

Ruta	Handler	Respuesta
<code>GET /health</code>	<code>app.ts</code>	<code>{ ok, service, uptimeSeconds, canvas: { host, mode: 'live', readOnly: true }, allowedEndpoints: [{ name, purpose }] }</code>
<code>GET /api/launch</code>	<code>launch.ts</code>	<code>{ courseId, courseName, userId, verified: false, source: 'config' 'enrollment' }</code> . Resuelve <code>users/self</code> (conserva solo el id) y el curso de <code>FARO_COURSE_ID</code> o la primera inscripción activa. Cachea en memoria. Token inválido → <code>502 canvas_token_rejected</code> .
<code>GET /canvas/api/v1/...</code>	<code>canvas.ts</code>	Lista blanca → <code>canvasGet</code> → JSON completo. Cabeceras <code>X-FARO-Canvas-Pages</code> y <code>X-FARO-Canvas-Endpoint</code> . Errores: <code>403</code> (no permitido), <code>401/403/404</code> (pasan de Canvas), <code>502 canvas_unreachable</code> , <code>504 canvas_timeout</code> .
<code>GET /lti/login</code> , <code>POST /lti/launch</code> , <code>GET /.well-known/jwks.json</code>	<code>lti.ts</code>	<code>501 lti_not_configured</code> con el paso que falta. La especificación completa del flujo está en los comentarios del archivo.

5.6 Arranque (`index.ts`)

```
[faro-server] listening on http://127.0.0.1:3000 -> Canvas canvas.instructure.com (read-only, token in memory only)
```

Sin `.env` válido:

```
[faro-server] CANVAS_ACCESS_TOKEN is required
[faro-server] Copy server/.env.example to server/.env and fill it in.
```

`SIGINT` / `SIGTERM` cierran el servidor limpiamente.

5.7 Pruebas (`test/`)

`fake-canvas.ts` es un Canvas de mentira que exige el Bearer, pagina con `Link` (2 filas por página para forzar varias páginas), y puede responder `401` en analíticas como hace Canvas con un estudiante sin permiso.

`e2e.ts` levanta el backend sobre ese Canvas y ejecuta 20 verificaciones. Se corre con `npm test` y termina con código distinto de cero al primer fallo. Lo que verifica es, literalmente, lo que el Libro 1 afirma.

5.8 Añadir una ruta de Canvas (procedimiento)

1. Justificarla: qué pantalla la necesita y qué dato mínimo lee.
2. Añadir la función en `src/data/canvas/endpoints.ts` y el scope en `requiredScopes`.
3. Añadir el patrón y su `purpose` en `server/src/allowlist.ts`.

4. Declarar solo los campos leídos en `src/data/canvas/raw.ts`.
5. Añadir el fixture en `fixtures.ts` y la ruta en `fixtureRoutes (client.ts)`.
6. Añadir la ruta al `fake-canvas.ts` y una verificación en `e2e.ts`.
7. Actualizar el Libro 1 (capítulo 2, inventario de datos) y este capítulo.

Si el paso 1 no tiene una respuesta clara, la ruta no se añade.

6. Implementación en Canvas, paso a paso: cuenta gratuita

Este procedimiento conecta FARO a un Canvas real en una tarde, sin pedir nada a una institución. Usa **Canvas Free-for-Teacher** (canvas.instructure.com), que es una instancia real de Canvas con la misma API.

Necesitas dos direcciones de correo: una para la cuenta *docente* (que crea el curso) y otra para la cuenta *estudiante* (la que FARO acompaña).

6.1 Crear el curso (cuenta docente) — 15 minutos

1. Entra a <https://canvas.instructure.com/register> y elige **Free for Teacher**. Completa el registro con el primer correo y confirma.
2. En el tablero, + **Course** (o *Start a new course*). Ponle nombre, por ejemplo *Gestión de Proyectos*, y **Create course**.
3. Crea al menos **dos módulos** (*Modules* → + *Module*). Dentro de cada uno, + → *Assignment* → *Create Assignment*, con nombre y puntos. Crea 4 o 5 actividades en total. Opcional: a alguna ponle fecha límite pasada para que FARO tenga algo *vencido* que reabrir.
4. **Publica** el curso (botón *Publish* en la página de inicio del curso) y **publica cada módulo y actividad** (el icono de nube en verde). Un ítem sin publicar no aparece para el estudiante.
5. Anota el **id del curso**: está en la URL, <https://canvas.instructure.com/courses/1234567> → `1234567`.

6.2 Inscribir al estudiante — 5 minutos

Dos caminos:

- **Invitación:** *People* → + *People*, escribe el segundo correo, rol *Student*, **Next** → **Add Users**. El estudiante recibe un correo y acepta.
- **Autoinscripción:** *Settings* → *Course Details* → *More Options* → *Let students self-enroll by sharing with them a secret URL*. Guarda. Aparece un enlace; el estudiante lo abre.

Con la cuenta estudiante (segundo correo), entra a Canvas, acepta la invitación, y **entrega al menos dos actividades** (aunque sea con un texto de una línea). Así FARO tiene progreso real que mostrar.

6.3 Generar el token (cuenta estudiante) — 2 minutos

Con la sesión del **estudiante**:

1. **Account** (arriba a la izquierda) → **Settings**.
2. Baja hasta **Approved Integrations** → + **New Access Token**.
3. *Purpose*: FARO *piloto*. *Expires*: pon una fecha (30 días es razonable). **Generate Token**.
4. **Copia el token ahora**. Canvas lo muestra completo solo en este momento.

Usa el token del **estudiante**, no del docente. Un token de docente no tiene *submission* propia ni *state* de módulo, y FARO mostraría un curso sin progreso. Además, limita el alcance de una fuga a lo que ese estudiante ve de sí mismo (Libro 1, capítulo 4).

6.4 Configurar y arrancar el backend — 3 minutos

```
cd server
cp .env.example .env
```

Edita `server/.env`:

```
CANVAS_API_URL=https://canvas.instructure.com
CANVAS_ACCESS_TOKEN=<pega aquí el token del estudiante>
FARO_COURSE_ID=1234567
```

Arranca:

```
npm start
```

Debes ver:

```
[faro-server] listening on http://127.0.0.1:3000 -> Canvas canvas.instructure.com (read-only, token in memory only)
```

Comprueba desde otra terminal:

```
curl http://127.0.0.1:3000/health
curl http://127.0.0.1:3000/api/launch
```

`/api/launch` debe devolver tu `courseId`, el nombre del curso y el `userId` numérico del estudiante. Si devuelve `canvas_token_rejected`, el token está mal copiado o expiró.

En Windows usa PowerShell: `Copy-Item .env.example .env` en vez de `cp`, y `Invoke-WebRequest http://127.0.0.1:3000/health | Select-Object -Expand Content` en vez de `curl` (o instala curl).

6.5 Construir la extensión en modo real — 3 minutos

En la raíz del proyecto crea `.env.local`:

```
VITE_CANVAS_MODE=http
VITE_FARO_API_URL=http://127.0.0.1:3000
```

Opción A, servidor de desarrollo (más rápido para iterar):

```
npm run dev
```

Abre `http://localhost:5173`. Ese origen ya está en `FARO_ALLOWED_ORIGINS` por defecto.

Opción B, extensión cargada en Chrome:

```
npm run build
```

Chrome → `chrome://extensions` → *Modo desarrollador* → *Cargar descomprimida* → carpeta `dist/`. Anota el **id de la extensión** que Chrome le asigna y añádelo a `server/.env`:

```
FARO_ALLOWED_ORIGINS=http://localhost:5173,chrome-extension://<id>
```

Reinicia el backend.

6.6 Verificar — 2 minutos

1. Abre FARO. Completa el propósito (tres preguntas).
2. **Inicio** debe mostrar el nombre real de tu curso y un progreso que coincida con las entregas del estudiante.
3. **Perfil** → **Canvas**: la insignia dice **En vivo**, aparece *Curso 1234567 · Gestión de Proyectos*, y la nota dice por qué backend pasan las llamadas. Abre el registro: cuatro llamadas con estado `200`.
4. En la terminal del backend, cuatro líneas de log con `status: 200` y **sin query strings ni token**.

Si en el registro la ruta `/analytics/.../activity` aparece con `401` y el panel muestra la nota sobre analíticas: es el comportamiento esperado para una cuenta de estudiante en Free-for-Teacher. FARO usa las fechas de tus entregas como actividad. No es un error.

6.7 Romperlo a propósito (opcional, recomendado)

- Revoca el token en Canvas (*Approved Integrations* → *eliminar*). Reinicia el backend: `canvas_token_rejected`. FARO muestra el error con *Reintentar*, no un curso vacío.
- Pide una ruta prohibida: `curl http://127.0.0.1:3000/canvas/api/v1/courses/1234567/enrollments` → `403 path_not_allowed`, y **ninguna línea nueva** en el log de Canvas: no llegó a la red.
- Cambia `FARO_ALLOWED_ORIGINS` para excluir `localhost:5173` y recarga el dev server: `403 origin_not_allowed`.

6.8 Problemas comunes

Síntoma	Causa probable	Solución
<code>CANVAS_API_URL</code> must use <code>https://</code>	Pusiste <code>http://</code>	Corrige la URL
<code>canvas_token_rejected</code>	Token mal copiado, expirado o revocado	Genera uno nuevo
<code>no_active_course</code>	El estudiante no aceptó la invitación o el curso no está publicado	Acepta la invitación / publica el curso
Inicio muestra 0 % con actividades	Las actividades no están en ningún módulo, o el token es del docente	Añade las actividades a módulos; usa token de estudiante
FARO no carga y la consola dice <code>CORS</code>	El origen de la extensión no está en <code>FARO_ALLOWED_ORIGINS</code>	Añade <code>chrome-extension://<id></code> y reinicia
<code>ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX</code>	Node anterior a 22.18	Actualiza Node
Módulos aparecen sin estado (<code>unlocked</code> todos)	Token de docente	Usa token de estudiante

7. Implementación en Canvas, paso a paso: institución con LTI 1.3

Este es el camino de despliegue real. Una extensión de navegador no se puede instalar institucionalmente; una herramienta LTI sí. Por eso el plan es **LTI 1.3 primero, la extensión como capa opcional** para quien la prefiera.

Lo que sigue tiene dos partes: lo que hace la institución (registrar la herramienta) y lo que hace el equipo de FARO (implementar tres rutas que hoy responden 501). Ninguna de las dos está hecha todavía; ambas están especificadas.

7.1 Qué pide FARO a la institución

Un administrador de Canvas crea una **Developer Key** → **LTI Key** con estos valores:

Campo	Valor
Method	Manual Entry
Title	FARO
Redirect URIs	<code>https://<faro>/lti/launch</code>
Target Link URI	<code>https://<faro>/lti/launch</code>
OpenID Connect Initiation Url	<code>https://<faro>/lti/login</code>
JWK Method	Public JWK URL: <code>https://<faro>/well-known/jwks.json</code>
LTI Advantage Services	☒ <code>contextmembership.readonly</code> (NRPS) · ☒ <code>result.readonly</code> (AGS) · ☐ <code>score</code> · ☐ <code>lineitem</code>
Placements	Course Navigation
Privacy Level	Anonymous o Public según decida la institución (ver 7.4)

Y una **Developer Key** → **API Key** con los *scopes* exactos de `src/data/canvas/endpoints.ts` (`requiredScopes`), con *Enforce Scopes* activado:

```
url:GET|/api/v1/courses/:id
url:GET|/api/v1/courses/:course_id/modules
url:GET|/api/v1/courses/:course_id/assignments
url:GET|/api/v1/courses/:course_id/analytics/users/:student_id/activity
url:GET|/api/v1/courses
url:GET|/api/v1/users/self
```

El administrador entrega a FARO: `client_id`, `deployment_id`, la URL de la instancia y la URL del JWKS de Canvas (`https://<institución>.instructure.com/api/lti/security/jwks`).

7.2 Qué implementa FARO (las tres rutas)

Todo con `node:crypto`, sin dependencias. La especificación está en `server/src/routes/lti.ts`.

GET /lti/login — inicio OIDC de terceros. Recibe `iss`, `login_hint`, `target_link_uri`, `client_id`, `lti_message_hint`. Genera `state` y `nonce` aleatorios, los guarda con expiración corta (memoria o base de datos), y redirige (302) a `https://<institución>.instructure.com/api/lti/authorize_redirect` con `scope=openid`, `response_type=id_token`, `response_mode=form_post`, `prompt=none`, `redirect_uri`, `client_id`, `login_hint`, `state`, `nonce`, `lti_message_hint`.

POST /lti/launch — verificación del `id_token`.

1. Compara `state` con el guardado; lo consume.
2. Decodifica el JWT; toma `kid` de la cabecera; obtiene la clave pública del JWKS de Canvas (cacheado); verifica la firma RS256 con `crypto.verify`.
3. Verifica `iss` (la instancia), `aud` (el `client_id`), `exp`, `iat`, y que `nonce` sea el emitido y no usado.
4. Lee `https://purl.imsglobal.org/spec/lti/claim/context` → `id` (curso) y `sub` (persona). Opcionalmente los endpoints de NRPS y AGS de sus *claims* respectivos.
5. Crea una sesión de FARO (cookie `HttpOnly`; `Secure`; `SameSite=None`) ligada a (curso, persona, instancia).
6. Redirige a la interfaz de FARO.

`GET /.well-known/jwks.json` — la clave pública de FARO. FARO genera un par RSA al desplegar (guardado como secreto), publica la pública aquí, y firma con la privada el `client_assertion` (RFC 7523) que envía a `https://<institución>.instructure.com/login/oauth2/token` con `grant_type=client_credentials` para obtener tokens de servicio LTI (NRPS/AGS).

7.3 Token de API por estudiante (OAuth2)

Para las seis rutas REST, con la API Key:

1. Tras el lanzamiento, si no hay token para esa persona, FARO redirige a `https://<institución>.instructure.com/login/oauth2/auth?client_id=...&response_type=code&redirect_uri=https://<faro>/oauth/callback&scope=<los seis scopes>&state=...`.
2. Canvas pide consentimiento al estudiante (una vez) y devuelve un `code`.
3. FARO lo canjea en `POST /login/oauth2/token` por `access_token` (1 hora) y `refresh_token`.
4. Ambos se guardan **solo en el backend**, cifrados en reposo, ligados a la persona.
5. Al expirar, FARO renueva con el `refresh_token`. Al retirar consentimiento, `DELETE /login/oauth2/token`.

Con esto desaparece `CANVAS_ACCESS_TOKEN` de `.env`: el backend pasa a tener un token por persona en vez de uno compartido.

7.4 Qué cambia en el backend

Hoy (piloto)	Producción
<code>GET /api/launch</code> deduce curso y persona del token en <code>.env</code>	Los lee de la sesión creada por <code>/lti/launch</code> ; <code>verified: true</code>
Un token para todo	Un token OAuth2 por persona, almacenado cifrado
Autorización por origen (CORS) y localhost	Autorización por sesión (cookie) + CORS
Estado de FARO en el navegador	PostgreSQL: propósito, sesiones, preferencias, tokens
Mentor local	<code>POST /api/mentor</code> en el backend, con el contexto de 11 campos y el contrato con el proveedor

`allowlist.ts`, `canvas.ts` y las rutas `/canvas/*` **no cambian**: la lista blanca y la paginación son las mismas con cualquier token.

Privacy Level de la LTI Key. *Anonymous* no envía nombre ni correo en el lanzamiento; FARO no los necesita, así que es la opción coherente con el Libro 1. *Public* los enviaría; FARO los ignoraría, pero mejor no recibirlos.

7.5 Orden de trabajo sugerido

1. Desplegar el backend detrás de TLS (dominio propio, certificado válido). Sin esto, Canvas no acepta la herramienta.
2. Generar el par de claves de FARO; publicar `/.well-known/jwks.json`.
3. Implementar `/lti/login` y `/lti/launch` contra un curso de prueba en la instancia de la institución (o en Free-for-Teacher, que también admite LTI Keys desde la cuenta docente).
4. Implementar el flujo OAuth2 de 7.3.
5. Sustituir el lanzamiento por sesión en `/api/launch`.
6. Añadir PostgreSQL y mover el estado de FARO.
7. Registrar la herramienta a nivel de cuenta; piloto con un curso.

7.6 La extensión en este escenario

Sigue existiendo como opción: quien prefiera el panel lateral en vez del iframe de Canvas la instala, y la extensión llama al mismo backend con la misma sesión (la cookie viaja si `FARO_ALLOWED_ORIGINS` incluye el id de la extensión y la cookie es `SameSite=None`). El código de la extensión no cambia; cambia solo dónde apunta `VITE_FARO_API_URL`.

8. Verificación y pruebas

8.1 Lo que se verifica automáticamente

`cd server && npm test` levanta un Canvas simulado con paginación real y el backend encima, y ejecuta estas 20 verificaciones:

```
ok health names the Canvas host and never the token
ok launch resolves the token owner and the first active course
ok course proxy returns the Canvas object
ok modules are paginated by Canvas and merged by the backend
ok assignments carry submissions across three pages
ok activity is forwarded for the token owner
ok a path outside the allow list is refused locally (no Canvas call)
ok a write is refused even on an allowed path
ok an unknown query parameter is refused
ok path traversal is refused
ok a browser origin outside the list gets 403 and no data
ok CORS preflight succeeds for an allowed origin
ok responses carry no-store and nosniff
ok Canvas 404 is passed through as 404
ok the token was sent to Canvas on every call, and only there
ok log lines carry no query strings
ok rate limit trips after the configured burst
ok Canvas 401 on analytics is passed through so the client can fall back
ok a wrong token is reported as rejected, not as a generic failure
ok .env parser handles comments, quotes and CRLF
```

20 checks passed

No usa ningún framework de pruebas: `node:assert` y un `check(nombre, fn)` de diez líneas. El proceso termina con código distinto de cero al primer fallo, así que sirve como paso de CI.

8.2 Lo que se verificó del mapeo de la extensión

Además de lo anterior, la capa `src/data/canvas/` se compiló y ejecutó en Node contra el mismo Canvas simulado y el backend real, en los cuatro escenarios que importan:

Escenario	Resultado esperado y obtenido
Mock	16 actividades, 7 completadas, 4 llamadas a fixtures, lanzamiento <code>fixture</code> , perfil de estudio presente
Http a través del backend	Curso GP-101, 3 módulos (el tercero <code>locked</code>), 5 actividades (la huérfana 999 excluida), 3 con entrega, <code>module_id</code> correcto, <code>course_id</code> del curso vivo, 20 min por defecto, 3 eventos de actividad ordenados del más reciente, <code>sin</code> perfil de estudio, <code>activityFallback = false</code> , 4 llamadas con 200
Analíticas prohibidas (401)	<code>activityFallback = true</code> , actividad derivada de 3 entregas, la ruta de analíticas con 401 en el registro, ninguna llamada a <code>/enrollments</code>
Backend caído	<code>getCourseSnapshot()</code> rechaza; nunca devuelve un curso vacío

Este script no está en el repositorio porque depende de compilar la extensión sin Vite; los escenarios sí están descritos aquí para reproducirlos.

8.3 Verificación manual en la interfaz

Con los fixtures (modo mock), la app debe mostrar exactamente:

Dato	Valor
Actividades totales	16
Completadas	7
Progreso	44 %
Restantes	9 actividades, 4 h 35 min
Días hasta el cierre	12
Días sin actividad	5
Estado de fricción	FRICTION

Dato	Valor
Módulo 3	iniciado
Módulo 5	bloqueado

Y en **Perfil** → **Canvas**: insignia *Simulado*, cuatro rutas GET, dos permisos rechazados, registro con cuatro entradas `200` (~140 ms).

8.4 Romperlo a propósito

Tres experimentos con los fixtures, cada uno con un resultado que debe cambiar:

1. **DAYS_INACTIVE de 5 a 9** → estado DISCONNECTION, Inicio cambia de tono, Recuperación ofrece misión de regreso. Nada de esto está escrito en las vistas: sale del umbral `disconnectionDays: 7`.
2. **DAYS_UNTIL_COURSE_ENDS de 12 a 3** → veredicto *no es realista* con las tres estrategias marcadas como no factibles y trabajo en desbordamiento.
3. **Una ruta con error de tipo en `endpoints.ts`** → 404 en el registro del panel. El mock no resuelve por coincidencia.

8.5 Typecheck

- Raíz: `npm run typecheck`. Requiere `npm install` previo (React, Vite). Si no imprime nada tras las dos líneas del encabezado, pasó.
- `server/`: `npm install && npm run typecheck`. Instala solo `typescript` y `@types/node` como dependencias de desarrollo; el backend en ejecución sigue sin dependencias.

`en.ts` tipado contra `es.ts` hace que una clave faltante en un idioma sea un error de compilación, no un texto en blanco en la demo.

8.6 Qué no está probado todavía

- La interfaz React contra un backend real en un navegador (las pruebas anteriores cubren la capa de datos, no el render). Se cubre con el procedimiento del capítulo 6.
- Las rutas LTI (responden 501 por diseño).
- Comportamiento con cursos grandes (más de 100 módulos o actividades, es decir, más de una página de Canvas real). La paginación está probada con el Canvas simulado; conviene repetirla contra Free-for-Teacher con `per_page=2` forzado en una prueba.

9. Qué es real y qué está simulado

Esta lista existe para que nadie —ni el jurado, ni una institución, ni el equipo dentro de seis meses— crea que el prototipo hace algo que no hace.

Real y funcionando

Componente	Estado
Backend con token, lista blanca, paginación, CORS, límite de tasa, logs sin datos sensibles	Implementado; 20 verificaciones automáticas
Extensión en modo <code>http</code> contra el backend, sin credenciales ni permiso hacia <code>instructure.com</code>	Implementado; capa de datos verificada end-to-end
Mapeo de Canvas a FARO (módulos por ítems, hecho por entrega, huérfanas excluidas)	Implementado y verificado con respuestas paginadas
Respaldo de actividad por entregas cuando Canvas niega analíticas, con aviso en pantalla	Implementado y verificado
Estado de error con reintento cuando el backend no responde	Implementado
Motor de fricción, ruta, siguiente mejor acción, planificador, puntos, logros, orden del feed, ruteo del SOS, contrato del mentor	Implementados como funciones puras; umbrales configurables
i18n es/en verificado por el compilador	Implementado
Tarjetas de reconexión	Flujo y reglas implementados; el banco de preguntas está escrito a mano para el curso del demo
Recompensas	Reglas implementadas (pista semestral, niveles, un canje por semestre); montos, cursos anteriores y canje simulados
Registro de peticiones en vivo en Perfil	Implementado

Listo para ejecutar, pendiente de credenciales

Componente	Qué falta
Conexión a Canvas Free-for-Teacher	Un token de estudiante y el id del curso (capítulo 6, 30 minutos)

Simulado, y etiquetado como tal

Componente	Qué falta
Datos del curso en modo mock	Vienen de <code>fixtures.ts</code> . La insignia dice <i>Simulado</i> .
Lanzamiento	<code>verified: false</code> en ambos modos. En vivo, curso y persona se deducen del token; el panel lo dice.
Cuenta institucional (SSO)	No existe. La pantalla lo etiqueta como simulado.
Comunidad	Compañeros, publicaciones, salas y misión son datos de ejemplo. Las salas corren con reloj acelerado para la demo.
Perfil de estudio (ritmo medido)	Solo en mock (35 min/sesión, 4 días/semana). En vivo, ausente y declarado.
<code>estimated_minutes</code>	No existe en Canvas. Tabla propia en mock; 20 min por defecto en vivo.
Mentor	Local y determinista, sin modelo. <code>HttpMentorService</code> existe en la extensión; <code>POST /api/mentor</code> no existe en el backend.

Diseñado, con la especificación en el código, no implementado

Componente	Dónde está la especificación
LTI 1.3: <code>/lti/login</code> , <code>/lti/launch</code> , <code>/.well-known/jwks.json</code>	<code>server/src/routes/lti.ts</code> ; capítulo 7
OAuth2 por estudiante	Capítulo 7.3; scopes en <code>endpoints.ts</code>
PostgreSQL y sincronización entre dispositivos	Instrucciones del proyecto; capítulo 7.4
Mentor remoto tras el backend	<code>services/mentor.ts</code> (<code>HttpMentorService</code>); Libro 1, capítulo 5
Advertencia en la frase de destino	Libro 1, capítulo 5.2

La regla de honestidad

Ninguna cifra de validación de este proyecto está inventada. Donde no hay medición, los documentos dicen *a medir*. Los umbrales de fricción son reglas de prototipo, configurables, y no se presentan como validadas científicamente. Lo que se afirma como implementado tiene código y, cuando es verificable, una prueba que se puede ejecutar.